

Assignment 3 - EECS 348 (Software Engineering) by Lucas Frias

Pleasure to have you read this assignment, as always. Today's will be difficult for me, as I am not one of the most frequent enjoyers of C++. Even still, I hope that I will be able to provide decent analysis.

On the model frontier, there was some news a couple weeks ago about KIMI3 by a Chinese-based company called Moonshot. Their new frontier model is open source, and very cheap because of that. Anthropic claims that their models were distilled and "stolen" by KIMI3. It is interesting, because we are seeing the fight between private and public development of LLMs and their future. I decided to use Moonshot's KIMI3.

Before the grader becomes apprehensive of me comparing like AI, I believe that Anthropic is not being fully transparent with their claims, and even if the claims are true, KIMI is different enough from just a pure copy paste model in the design choices it implemented. These are separate models.

For Claude, I did the same method as before. Our returner will be Sonnet (on High reasoning). I was able to access this model for signing up for a free account on Claude.ai, and then prompted it the prompt in PROMPT.md by copy and pasting the text. I have gotten a Claude Pro subscription between this assignment and the next, but I do not think it overly impacted the reasoning, as I used the website version (no access to the terminal or other tools not already provided to it by Anthropic). The reasoning depths might have been increased slightly, however.

To access KIMI, I had to use a Google account to sign up for the online portal after denying my U.S. based phone number. After jumping through this hurdle, the website worked about as much as any other chat bot would. I copy and pasted the text into the prompt. Here's the prompt I used for both of them:

Make this program in C++. You have the added requirement to code using objects, not functions.

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

Here's the file format:

EMAIL <sender category>, <subject line>, <date> - The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next

- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY

NEXT - Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:
Subject:
Date:

READ - CEO has read with the email and has dealt with it

COUNT - display current count of untracked files

Here's an example file:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Here is the output that the program should give
There are 6 emails to read.

Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025

Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024

There are 4 emails to read.

Good luck!

Formatting with backticks was removed in this copy and paste, but otherwise this is the identical prompt.

Output

Anthropic's Sonnet's Output

```
// ceo_inbox.cpp
// prioritizes emails for a ceo using a hand-built max heap.
// build: g++ -std=c++11 -Wall -Wextra -o ceo_inbox ceo_inbox.cpp
// run: ./ceo_inbox inbox.txt

#include <fstream>
#include <iostream>
#include <string>

// -----
// email: one message. knows how to rank itself.
// -----
class Email {
private:
    std::string sender;
    std::string subject;
    std::string date;      // MM-DD-YYYY
    long long sequence;    // arrival order, breaks exact ties (earlier wins)

    // higher rank = read sooner
    static int RankOf(const std::string& category) {
        if (category == "Boss")          return 5;
        if (category == "Subordinate")    return 4;
        if (category == "Peer")           return 3;
        if (category == "ImportantPerson") return 2;
        if (category == "OtherPerson")    return 1;
        return 0; // unknown
    }

    // MM-DD-YYYY -> YYYYMMDD so a bigger number means a newer date
    static long DateKeyOf(const std::string& d) {
        if (d.size() != 10 || d[2] != '-' || d[5] != '-') return 0;
        int month = std::stoi(d.substr(0, 2));
        int day   = std::stoi(d.substr(3, 2));
        int year  = std::stoi(d.substr(6, 4));
        return static_cast<long>(year) * 10000L + month * 100L + day;
    }

public:
    Email() : sender(""), subject(""), date(""), sequence(0) {}
};
```

```

Email(const std::string& s, const std::string& subj,
      const std::string& d, long long seq)
    : sender(s), subject(subj), date(d), sequence(seq) {}

const std::string& GetSender() const { return sender; }
const std::string& GetSubject() const { return subject; }
const std::string& GetDate() const { return date; }

static bool IsValidCategory(const std::string& category) {
    return RankOf(category) > 0;
}

static bool IsValidDate(const std::string& d) {
    if (d.size() != 10 || d[2] != '-' || d[5] != '-') return false;
    for (std::size_t i = 0; i < d.size(); ++i) {
        if (i == 2 || i == 5) continue;
        if (d[i] < '0' || d[i] > '9') return false;
    }
    return true;
}

// true when this email must be read before the other one
bool HasHigherPriorityThan(const Email& other) const {
    int mine    = RankOf(sender);
    int theirs  = RankOf(other.sender);
    if (mine != theirs) return mine > theirs;

    long myDate    = DateKeyOf(date);
    long theirDate = DateKeyOf(other.date);
    if (myDate != theirDate) return myDate > theirDate; // newest first

    return sequence < other.sequence; // earlier arrival
}

first
}
};

// -----
// maxheap: list-based (array) binary heap. built from scratch.
// parent of i is (i-1)/2. children of i are 2i+1 and 2i+2.
// -----
class MaxHeap {
private:
    Email*      items;
    std::size_t count;
    std::size_t capacity;

    void Grow() {
        std::size_t newCapacity = capacity * 2;

```

```

        Email* bigger = new Email[newCapacity];
        for (std::size_t i = 0; i < count; ++i) bigger[i] = items[i];
        delete[] items;
        items = bigger;
        capacity = newCapacity;
    }

    void Swap(std::size_t a, std::size_t b) {
        Email temp = items[a];
        items[a] = items[b];
        items[b] = temp;
    }

    void SiftUp(std::size_t index) {
        while (index > 0) {
            std::size_t parent = (index - 1) / 2;
            if (items[index].HasHigherPriorityThan(items[parent])) {
                Swap(index, parent);
                index = parent;
            } else {
                break;
            }
        }
    }

    void SiftDown(std::size_t index) {
        while (true) {
            std::size_t left = 2 * index + 1;
            std::size_t right = 2 * index + 2;
            std::size_t largest = index;

            if (left < count &&
items[left].HasHigherPriorityThan(items[largest]))
                largest = left;
            if (right < count &&
items[right].HasHigherPriorityThan(items[largest]))
                largest = right;

            if (largest == index) break;
            Swap(index, largest);
            index = largest;
        }
    }

public:
    MaxHeap() : items(new Email[8]), count(0), capacity(8) {}
    ~MaxHeap() { delete[] items; }

    // a raw owned array makes copying unsafe. forbid it.

```

```

MaxHeap(const MaxHeap&) = delete;
MaxHeap& operator=(const MaxHeap&) = delete;

bool IsEmpty() const { return count == 0; }
std::size_t Size() const { return count; }

void Insert(const Email& email) {
    if (count == capacity) Grow();
    items[count] = email;
    SiftUp(count);
    ++count;
}

// caller must check IsEmpty() first
const Email& Peek() const { return items[0]; }

// caller must check IsEmpty() first
Email ExtractMax() {
    Email top = items[0];
    --count;
    if (count > 0) {
        items[0] = items[count];
        SiftDown(0);
    }
    return top;
}
};

// -----
// inboxprocessor: owns the heap, reads the command file, prints results.
// -----
class InboxProcessor {
private:
    MaxHeap heap;
    long long nextSequence;

    static std::string Trim(const std::string& text) {
        const std::string whitespace = " \t\r\n";
        std::size_t first = text.find_first_not_of(whitespace);
        if (first == std::string::npos) return "";
        std::size_t last = text.find_last_not_of(whitespace);
        return text.substr(first, last - first + 1);
    }

    void HandleEmail(const std::string& fields) {
        std::size_t firstComma = fields.find(',');
        std::size_t secondComma = (firstComma == std::string::npos
                                   ? std::string::npos
                                   : fields.find(',', firstComma + 1));
    }
};

```

```

        if (secondComma == std::string::npos) {
            std::cerr << "skipped bad EMAIL line (need 3 fields): " << fields <<
"\n";
            return;
        }

        std::string sender = Trim(fields.substr(0, firstComma));
        std::string subject = Trim(fields.substr(firstComma + 1,
                                                    secondComma - firstComma - 1));
        std::string date = Trim(fields.substr(secondComma + 1));

        if (!Email::IsValidCategory(sender)) {
            std::cerr << "skipped EMAIL with unknown sender: " << sender << "\n";
            return;
        }
        if (!Email::IsValidDate(date)) {
            std::cerr << "skipped EMAIL with bad date: " << date << "\n";
            return;
        }

        heap.Insert(Email(sender, subject, date, nextSequence++));
    }

    void HandleCount() const {
        std::cout << "There are " << heap.Size() << " emails to read.\n\n";
    }

    void HandleNext() const {
        if (heap.IsEmpty()) {
            std::cout << "No emails to read.\n\n";
            return;
        }
        const Email& top = heap.Peek();
        std::cout << "Next email:\n"
            << "Sender: " << top.GetSender() << "\n"
            << "Subject: " << top.GetSubject() << "\n"
            << "Date: " << top.GetDate() << "\n\n";
    }

    void HandleRead() {
        if (heap.IsEmpty()) {
            std::cout << "No emails to read.\n\n";
            return;
        }
        heap.ExtractMax();
    }

public:
    InboxProcessor() : heap(), nextSequence(0) {}

```

```

bool ProcessFile(const std::string& path) {
    std::ifstream input(path.c_str());
    if (!input.is_open()) {
        std::cerr << "cannot open file: " << path << "\n";
        return false;
    }

    std::string line;
    while (std::getline(input, line)) {
        line = Trim(line);
        if (line.empty()) continue;

        std::size_t space = line.find(' ');
        std::string command = (space == std::string::npos) ? line
                                                                : line.substr(0,
space);
        std::string rest = (space == std::string::npos) ? ""
                                                                :
line.substr(space + 1);

        if (command == "EMAIL")    HandleEmail(rest);
        else if (command == "COUNT") HandleCount();
        else if (command == "NEXT") HandleNext();
        else if (command == "READ") HandleRead();
        else std::cerr << "skipped unknown command: " << line << "\n";
    }
    return true;
}

};

// -----
// application: entry point wrapper so main holds no logic.
// -----
class Application {
public:
    int Run(int argc, char* argv[]) {
        std::string path;
        if (argc > 1) {
            path = argv[1];
        } else {
            std::cout << "input file: ";
            std::getline(std::cin, path);
        }

        InboxProcessor processor;
        return processor.ProcessFile(path) ? 0 : 1;
    }
};

```

```

int main(int argc, char* argv[]) {
    Application app;
    return app.Run(argc, argv);
}

```

Kimi 3's Output:

```

#include <iostream>
#include <fstream>
#include <string>
#include <cstdio>
using namespace std;

// ----- Date -----
class Date {
public:
    int month, day, year;
    Date() : month(0), day(0), year(0) {}
    // Build a date from an "MM-DD-YYYY" string
    Date(const string& s) {
        month = stoi(s.substr(0, 2));
        day   = stoi(s.substr(3, 2));
        year  = stoi(s.substr(6, 4));
    }
    // Numeric key so newer dates compare greater
    int key() const { return year * 10000 + month * 100 + day; }
    string str() const {
        char buf[11];
        snprintf(buf, sizeof(buf), "%02d-%02d-%04d", month, day, year);
        return string(buf);
    }
};

// ----- Email -----
class Email {
public:
    string sender;
    string subject;
    Date date;

    Email() {}

    // Parse one EMAIL line payload: "<sender>, <subject>, <date>"
    Email(const string& payload) {
        size_t c1 = payload.find(',');
        size_t c2 = payload.rfind(',');
        sender    = payload.substr(0, c1);
        subject    = payload.substr(c1 + 1, c2 - c1 - 1);
        date       = Date(payload.substr(c2 + 1));
    }
};

```

```

    }

    int priority() const {
        if (sender == "Boss")           return 5;
        if (sender == "Subordinate")     return 4;
        if (sender == "Peer")            return 3;
        if (sender == "ImportantPerson") return 2;
        return 1; // OtherPerson
    }

    // true if THIS email should be read before e
    bool higherThan(const Email& e) const {
        if (priority() != e.priority())
            return priority() > e.priority();
        return date.key() > e.date.key(); // newest first
    }
};

// ----- Heap node -----
class HeapNode {
public:
    Email data;
    HeapNode *parent, *left, *right;
    explicit HeapNode(const Email& e)
        : data(e), parent(nullptr), left(nullptr), right(nullptr) {}
};

// ----- MaxHeap (list-based) -----
class MaxHeap {
    HeapNode* root;
    int count;

    void clear(HeapNode* n) {
        if (!n) return;
        clear(n->left);
        clear(n->right);
        delete n;
    }

    // Locate the node at 1-based position n by following the
    // binary representation of n (drop leading 1; 0 = left, 1 = right)
    HeapNode* nodeAt(int n) const {
        HeapNode* cur = root;
        int bits = 31 - __builtin_clz((unsigned)n); // index of leading bit
        for (int b = bits - 1; b >= 0; --b) {
            cur = ((n >> b) & 1) ? cur->right : cur->left;
            if (!cur) break;
        }
        return cur;
    }
};

```

```

    }

    void siftUp(HeapNode* n) {
        while (n->parent && n->data.higherThan(n->parent->data)) {
            swap(n->data, n->parent->data);
            n = n->parent;
        }
    }

    void siftDown(HeapNode* n) {
        while (n) {
            HeapNode* best = n;
            if (n->left && n->left->data.higherThan(best->data)) best = n-
>left;
            if (n->right && n->right->data.higherThan(best->data)) best = n-
>right;
            if (best == n) break;
            swap(n->data, best->data);
            n = best;
        }
    }

public:
    MaxHeap() : root(nullptr), count(0) {}
    ~MaxHeap() { clear(root); }

    int size() const { return count; }
    bool empty() const { return count == 0; }

    void insert(const Email& e) {
        HeapNode* node = new HeapNode(e);
        ++count;
        if (!root) { root = node; return; }
        // Attach at position 'count' (the next complete-tree slot)
        HeapNode* pos = nodeAt(count / 2);           // parent of new slot
        if (count % 2 == 0) pos->left = node; else pos->right = node;
        node->parent = pos;
        siftUp(node);
    }

    const Email& peekMax() const { return root->data; }

    Email extractMax() {
        Email top = root->data;
        HeapNode* last = nodeAt(count);               // last node in tree
        --count;
        if (last == root) {
            delete root;
            root = nullptr;
        }
    }

```

```

        return top;
    }
    root->data = last->data;           // move last data to root
    if (last->parent->left == last) last->parent->left = nullptr;
    else last->parent->right = nullptr;
    delete last;
    siftDown(root);
    return top;
}
};

// ----- main -----
int main(int argc, char* argv[]) {
    string filename;
    if (argc > 1) filename = argv[1];
    else { cout << "Enter input file name: "; cin >> filename; }

    ifstream in(filename);
    if (!in) { cerr << "Cannot open file: " << filename << endl; return 1; }

    MaxHeap inbox;
    string line;
    while (getline(in, line)) {
        // trim trailing whitespace / CR
        while (!line.empty() && isspace((unsigned char)line.back()))
line.pop_back();
        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            inbox.insert(Email(line.substr(6)));
        } else if (line == "COUNT") {
            cout << "There are " << inbox.size() << " emails to read.\n\n";
        } else if (line == "NEXT") {
            if (inbox.empty()) {
                cout << "No emails to read.\n\n";
            } else {
                const Email& e = inbox.peekMax();
                cout << "Next email: \n"
                     << "Sender: " << e.sender << "\n"
                     << "Subject: " << e.subject << "\n"
                     << "Date: " << e.date.str() << "\n\n";
            }
        } else if (line == "READ") {
            if (!inbox.empty()) inbox.extractMax();
        }
    }
    return 0;
}

```

Correctness

I was reasonably impressed on a first inspection with the output of the two in terms of correctness. In fact, they output the exact same output. However, internally they function differently. We will discuss this later. Just to be said that both are “correct”.

As per the main requirement, they use objects and methods within the objects instead of functions. They also implement their own “list” based implementation, as the rubric given on Canvas states:

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

But both of them did this tremendously differently. While Sonnet implemented a similar method to what it had in C (using a heap allocated array which it dynamically grows) KIMI used a linked list!! This was shocking to me. I mentioned it as a possibility in Assignment 2 previously as a slightly impractical but interesting solution, and the reason why was the cost of lookup not being $O(1)$, but is $O(\log(N))$. We’ll discuss whether this was a decent choice or not in the next section.

In terms of handling incorrect user input, only Sonnet implements actual error handling, which will fail to add the element to the maxheap with standard output. KIMI will fail to process the element correctly and have all sorts of errors if the field isn’t processed correctly and is malformed.

Sonnet will not do realistic date formatting though. It will format the year, month, or day, incorrectly if it goes above 12, 31, or some unrealistic year within 4 digits. This is because it doesn’t implement basic error checking for those dates. This is kind of to be expected, and the issue seems to be more on the representation of the data, too. I would give Sonnet a “functional” evaluation, but the code is not completely airtight when we don’t assume nice data, while KIMI doesn’t work well at all if it’s any bad data.

One last comment: as much as I have distaste in C++, I have to concede that the C++ implementation is more legible and better when implementing complex data structures. Reading takes much less mental effort. Both Sonnet and Kimi implemented commonsense implementations that used C++ features (for example, the better string library) much better than I could have

Given test inputs generated by me and my test file generator in Python, I found that the programs were functional identically. However, under the hood there’s a real distance. Let’s discuss that.

Time Complexity

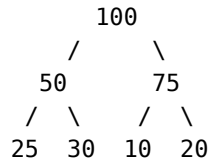
This is where my analysis becomes difficult:

In terms of time complexity, there is the theoretical versus real difference. The theoretical section will discuss Sonnet versus Kimi, whereas the real section will discuss input and scaling.

Theoretical

As is probably common knowledge, a traditional maxheap implementation is done in an array. This is done for one main reason: unlike a binary search or even a normal binary tree, there are certain guarantees that we can make about a max heap, which are: 1. The top will always be the maximum value 2. Going down the tree will lead to smaller values 3. For any complete given level i , there will always be 2^i elements in each level

This allows us to represent a MaxHeap like so:



Since we really care about peaking the max value, we can represent the tree like this, because searching for a given element doesn't particularly matter.

This gives us $O(n)$ space, as expected, but also allows us to represent data in a contiguous array like so:

```
int max_heap[7] = {100, 50, 75, 25, 30, 10, 20};
```

Because of the nature of this tree, every node to the left, given a node's position denoted by i , is $2i+1$ and $2i+2$. We can get our parent node by doing $((i-1) \gg 1)$.

This gives us very fast lookup times because these elements are right next to one another, but even more critically lowers the compute time used to access and load each element. Preserving the original structure becomes much more simple because they become array operations (example, upshifting is just swapping two array values from each of their respective positions to one another).

Typically, one would program this with an array. This was the Sonnet implementation that was used to compute the array before, along with Luna. Sonnet does this (with the heap allocation keyword "new" during its implementation specifications. However, KIMI decides to go completely south of this. KIMI uses a linked list implementation.

I wonder if the wording using "list" and this model being a Chinese based model had a play in this very specifically being used, because there are some disadvantages

The biggest issue with Kimi's implementation is that, because we are using a node based Max-Heap, the heap does not have an $O(1)$ time complexity for member lookup, but a $O(32)$? lookup.

```
HeapNode* nodeAt(int n) const {
    HeapNode* cur = root;
    int bits = 31 - __builtin_clz((unsigned)n); // index of leading bit
    for (int b = bits - 1; b >= 0; --b) {
        cur = ((n >> b) & 1) ? cur->right : cur->left;
        if (!cur) break;
    }
}
```

```

    }
    return cur;
}

```

This code is fairly complex, but from what I understand, the maximum amount of iteration is around 30 as this is the max size of int (4 bytes). But instead of using pointer arithmetic such as `base + offset` to find the value (which Sonnet can do because the values are contiguous, but KIMI cannot) it must iterate through a finite but large amount of memory, potentially cold reloading if there becomes thousands upon thousands of emails.

The bizarre theoretical choices don't stop there, as the same problems with locality and speed become important when discussing upheaping and downheaping. The space complexity might be a little more germane, but again, if the array is contiguous it can be loaded into chunks and into a hot cache (for example, L1), instead of a more erratic spacing. Upheaping and downheaping with arrays have the added benefit that for every element, especially closer to the tree, the nearest element is only $(i-1) \gg 1$ away from the previous element.

The added reason to not do this is the unnecessary complexity, but we'll discuss that more in Maintainability.

Real

The question then becomes, does this matter? When executing through the program, is there a noticable slowdown from KIMI to Sonnet, especially with very large entries, or is this just nonsense and theoretical observations?

Here's the output:

```

l382f965@cycle4:~/Code/SoftwareEng/
Fall2026_EECS348_Assignment_3/3195413_Assignment_3/exec/x86-KUdebian$ time ./
anthropic
input file: BIG.txt
There are 2999 emails to read.

```

```

real    0m2.360s
user    0m0.013s
sys 0m0.001s
l382f965@cycle4:~/Code/SoftwareEng/
Fall2026_EECS348_Assignment_3/3195413_Assignment_3/exec/x86-KUdebian$ time ./
moonshot
Enter input file name: BIG.txt
There are 2999 emails to read.

```

```

real    0m2.461s
user    0m0.003s
sys 0m0.004s
l382f965@cycle4:~/Code/SoftwareEng/
Fall2026_EECS348_Assignment_3/3195413_Assignment_3/exec/x86-KUdebian$

```

There is a difference (and a ~100 ms one too) between moonshot and anthropic's output. I had Claude Opus 5.5 make a linear function that shows the growth of their time complexity through this Python script here:

```
#!/usr/bin/env python3
"""Benchmark ./anthropic and ./moonshot on growing email inputs and fit a curve.
```

```
For each input size from 10 to 10,000 emails:
```

1. write an input file of EMAIL lines followed by COUNT
2. run `time ./program < file` in bash for each program
3. log the real/user/sys times to timings.csv

```
Then, for each program, fit several growth models to the real times, pick the
best one, and print it as a Python function.
```

```
Run it from the folder that contains ./anthropic and ./moonshot:
```

```
python3 benchmark.py
"""
```

```
import csv
import datetime
import math
import os
import random
import statistics
import subprocess

PROGRAMS = ["../anthropic", "./moonshot"]
MIN_EMAILS = 10
MAX_EMAILS = 10_000
NUM_SIZES = 25          # how many input sizes between MIN and MAX
REPEATS = 3             # runs per size; the median is used
INPUT_DIR = "inputs"
LOG_FILE = "timings.csv"

SENDERS = ["Boss", "ImportantPerson", "Subordinate", "Peer", "OtherPerson"]
SUBJECT_WORDS = ["Important", "Meeting", "Update", "Report", "Budget", "Review",
                 "Lunch", "Deadline", "Reminder", "Urgent", "Project", "Policy",
                 "Invoice", "Schedule", "Follow", "Up", "Status", "Travel"]
DATE_START = datetime.date(2023, 1, 1)
DATE_SPAN = (datetime.date(2026, 12, 31) - DATE_START).days

# ----- input generation -----

def random_email():
    sender = random.choice(SENDERS)
    subject = " ".join(random.choices(SUBJECT_WORDS, k=random.randint(1, 3)))
    date = DATE_START + datetime.timedelta(days=random.randint(0, DATE_SPAN))
```

```

    return f"EMAIL {sender},{subject},{date:%m-%d-%Y}"

def write_input(n):
    os.makedirs(INPUT_DIR, exist_ok=True)
    path = os.path.join(INPUT_DIR, f"emails_{n}.txt")
    with open(path, "w") as f:
        for _ in range(n):
            f.write(random_email() + "\n")
        f.write("COUNT\n")
    return path

def sizes():
    """Log-spaced sizes from MIN_EMAILS to MAX_EMAILS, so small and large n are
    both covered."""
    ratio = (MAX_EMAILS / MIN_EMAILS) ** (1 / (NUM_SIZES - 1))
    out = sorted({round(MIN_EMAILS * ratio ** i) for i in range(NUM_SIZES)})
    out[-1] = MAX_EMAILS
    return out

# ----- timing -----

def time_program(program, input_path):
    """Run `echo input_path | time program` in bash and return (real, user, sys)
    in seconds.

    The program reads the file *name* from stdin and prints emails as it runs,
    so its stdout and stderr are both thrown away. Bash's `time` keyword reports
    on the shell's stderr, not the program's, so the timing line still comes
    through.
    """
    # TIMEFORMAT makes bash's `time` print just "real user sys" so it is easy to
    parse.
    cmd = (f"TIMEFORMAT='%R %U %S'; "
           f"time (echo '{input_path}' | {program} > /dev/null 2>&1)")
    result = subprocess.run(["bash", "-c", cmd], capture_output=True, text=True)
    if result.returncode != 0:
        raise RuntimeError(f"{program} exited with code {result.returncode} on
        {input_path}. "
                           f"Try it by hand: echo {input_path} | {program}")
    real, user, sys_ = result.stderr.strip().splitlines()[-1].split()
    return float(real), float(user), float(sys_)

# ----- curve fitting (no numpy needed) -----

def fit_line(xs, ys):

```

```

    """Least squares for  $y = a + b \cdot x$ . Returns (a, b)."""
    mx, my = statistics.fmean(xs), statistics.fmean(ys)
    sxx = sum((x - mx) ** 2 for x in xs)
    if sxx == 0:
        return my, 0.0
    b = sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / sxx
    return my - b * mx, b

def sse(ys, preds):
    return sum((y - p) ** 2 for y, p in zip(ys, preds))

# Each model is  $y = a + b \cdot g(n)$ . The name is how it prints.
MODELS = {
    "log n": (lambda n: math.log(n), "math.log(n)"),
    "n": (lambda n: n, "n"),
    "n log n": (lambda n: n * math.log(n), "n * math.log(n)"),
    "n^2": (lambda n: n ** 2, "n ** 2"),
    "n^3": (lambda n: n ** 3, "n ** 3"),
}

def fit_models(ns, ys):
    """Fit every model and return a list of (aic, name, a, b, k, expr), best
    first.

    AIC penalizes extra parameters, so the free power law  $n^k$  only wins if it
    fits clearly better than the fixed shapes.
    """
    results = []
    m = len(ns)

    def aic(err, params):
        return m * math.log(max(err, 1e-300) / m) + 2 * params

    # Constant:  $y = a$ 
    a = statistics.fmean(ys)
    results.append((aic(sse(ys, [a] * m), 1), "constant", a, 0.0, None, f"{a!r}"))

    for name, (g, expr) in MODELS.items():
        xs = [g(n) for n in ns]
        a, b = fit_line(xs, ys)
        if b < 0:
            continue # running time shouldn't shrink as input grows
        err = sse(ys, [a + b * x for x in xs])
        results.append((aic(err, 2), name, a, b, None, f"{a!r} + {b!r} * {expr}"))

```

```

# Free power law:  $y = a + b * n^k$ , grid search over k
best = None
for i in range(10, 401):
    k = i / 100
    xs = [n ** k for n in ns]
    a, b = fit_line(xs, ys)
    if b < 0:
        continue
    err = sse(ys, [a + b * x for x in xs])
    if best is None or err < best[0]:
        best = (err, a, b, k)
if best:
    err, a, b, k = best
    results.append((aic(err, 3), f"n^{k:.2f}", a, b, k, f"{a!r} + {b!r} * n
** {k}"))

results.sort(key=lambda r: r[0])
return results

def r_squared(ns, ys, func):
    my = statistics.fmean(ys)
    ss_tot = sum((y - my) ** 2 for y in ys)
    ss_res = sse(ys, [func(n) for n in ns])
    return 1 - ss_res / ss_tot if ss_tot else 1.0

# ----- main -----

def main():
    random.seed(210)
    for program in PROGRAMS:
        if not os.access(program, os.X_OK):
            raise SystemExit(f"Can't run {program}: make sure it exists here and
is executable.")

    rows = []
    with open(LOG_FILE, "w", newline="") as f:
        log = csv.writer(f)
        log.writerow(["program", "emails", "run", "real", "user", "sys"])
        for n in sizes():
            path = write_input(n)
            for program in PROGRAMS:
                reals = []
                for run in range(1, REPEATS + 1):
                    real, user, sys_ = time_program(program, path)
                    log.writerow([program, n, run, real, user, sys_])
                f.flush()

```

```

        reals.append(real)
    median = statistics.median(reals)
    rows.append((program, n, median))
    print(f"{program:12} n={n:>6} real={median:.3f}s")

print(f"\nRaw timings logged to {LOG_FILE}\n")

for program in PROGRAMS:
    ns = [n for p, n, _ in rows if p == program]
    ys = [t for p, _, t in rows if p == program]
    fits = fit_models(ns, ys)
    _, name, *_ , expr = fits[0]
    func_name = os.path.basename(program) + "_time"
    source = f"def {func_name}(n):\n    return {expr}\n"
    namespace = {"math": math}
    exec(source, namespace)
    r2 = r_squared(ns, ys, namespace[func_name])

    print(f"=== {program} ===")
    print(f"Best fit: 0({name})    R^2 = {r2:.4f}")
    print("Runner-ups: " + ", ".join(f"0({r[1]})" for r in fits[1:4]))
    print(source)

try:
    import matplotlib.pyplot as plt
except ImportError:
    print("(Install matplotlib to also get a plot: pip install matplotlib)")
    return

for program in PROGRAMS:
    ns = [n for p, n, _ in rows if p == program]
    ys = [t for p, _, t in rows if p == program]
    expr = fit_models(ns, ys)[0][5]
    func = eval(f"lambda n: {expr}", {"math": math})
    smooth = list(range(MIN_EMAILS, MAX_EMAILS + 1, 50))
    line, = plt.plot(ns, ys, "o", label=f"{program} measured")
    plt.plot(smooth, [func(n) for n in smooth], "--", color=line.get_color(),
             label=f"{program} fit")
plt.xlabel("emails")
plt.ylabel("real time (s)")
plt.legend()
plt.savefig("timings.png", dpi=150)
print("Plot saved to timings.png")

if __name__ == "__main__":
    main()

```

and got this output:

```

=== ./anthropic ===
Best fit: O(n)    R^2 = 0.9845
Runner-ups: O(n^0.99), O(n log n), O(n^2)
def anthropic_time(n):
    return 0.0017779290048449987 + 2.018834651340972e-06 * n

=== ./moonshot ===
Best fit: O(n log n)    R^2 = 0.9678
Runner-ups: O(n^1.09), O(n), O(n^2)
def moonshot_time(n):
    return 0.0018221431602645339 + 1.225946836190034e-07 * n * math.log(n)

```

This demonstrates a serious performance cost. Especially when scaling large input, $O(n \log(n))$ is significantly worse. This is because the memory is not contiguous, and makes the implementation much slower. I theorize that the data doesn't have good cache locality, so when executing, there is a significant delay in loading every element instead of being able to access it closer. We'll discuss this more in Space Complexity, but this hardware limitation makes this implementation much worse than what it should and needs to be. For millions or even just thousands of inputs, Kimi scales poorly.

There isn't too much else to be said. The implementation cost is not significant enough, though newer memory is much quicker and larger. If this was legacy hardware, there would be a significant visual demonstration of the costs incurred by Kimi compared to Sonnet. On a modern x86 Intel server such as KU's cycle server, the cost is less visible but still transparent through the simulation implemented above.

Space Complexity

Space complexity is a more interesting story. This is something that the previous LLMs struggled with repeating struct fields, but it seems as if these haven't made the same basic mistakes. Even still, the question remains onto whether the differing approaches are better for space complexity as they are for time complexity.

Theoretical

KIMI's choice is, theoretically, better for space complexity. This is because of the way that theoretically the space is allocated.

For every single element in a heap allocated array, we need to allocate a fixed number of space for our elements. But if the array grows, we have to reallocate another amount of space (specifically, Sonnet does *2new_capacity as the allocation*). *So the space complexity grows extra than what we need. Say we have a capacity of 10 and want to add one last element. This means we have to allocated 2 and use an extra 9 elements of space that is wasted. And if elements are freed, the space doesn't shrink dynamically. It remains the way it is before the allocation occurs.*

Meanwhile, Kimi uses nodes, which are only allocated on an as-needed basis, and freed when removed, which allows for the space to be resulting in the exact N number of elements that occur

within the data structure. The question is, does this implementation actually translate to a better space complexity.

Real

```
#!/usr/bin/env python3
"""Benchmark ./anthropic and ./moonshot on growing email inputs and fit curves.
```

For each input size from 10 to 10,000 emails:

1. write an input file of EMAIL lines followed by COUNT
2. run `echo file | ./program` for each program, with its output silenced
3. log real/user/sys time and peak memory (max RSS) to timings.csv

Then, for each program, fit several growth models to the real time and to the peak memory, pick the best one for each, and print them as Python functions.

Run it from the folder that contains ./anthropic and ./moonshot:

```
python3 benchmark.py
"""
```

```
import csv
import datetime
import math
import os
import random
import statistics
import subprocess
import sys
import time
```

```
PROGRAMS = ["./anthropic", "./moonshot"]
MIN_EMAILS = 10
MAX_EMAILS = 10_000
NUM_SIZES = 25          # how many input sizes between MIN and MAX
REPEATS = 3             # runs per size; the median is used
INPUT_DIR = "inputs"
LOG_FILE = "timings.csv"
```

```
SENDERS = ["Boss", "ImportantPerson", "Subordinate", "Peer", "OtherPerson"]
SUBJECT_WORDS = ["Important", "Meeting", "Update", "Report", "Budget", "Review",
                 "Lunch", "Deadline", "Reminder", "Urgent", "Project", "Policy",
                 "Invoice", "Schedule", "Follow", "Up", "Status", "Travel"]
DATE_START = datetime.date(2023, 1, 1)
DATE_SPAN = (datetime.date(2026, 12, 31) - DATE_START).days
```

```
# ----- input generation -----
```

```
def random_email():
    sender = random.choice(SENDERS)
```

```

subject = " ".join(random.choices(SUBJECT_WORDS, k=random.randint(1, 3)))
date = DATE_START + datetime.timedelta(days=random.randint(0, DATE_SPAN))
return f"EMAIL {sender},{subject},{date:%m-%d-%Y}"

def write_input(n):
    os.makedirs(INPUT_DIR, exist_ok=True)
    path = os.path.join(INPUT_DIR, f"emails_{n}.txt")
    with open(path, "w") as f:
        for _ in range(n):
            f.write(random_email() + "\n")
        f.write("COUNT\n")
    return path

def sizes():
    """Log-spaced sizes from MIN_EMAILS to MAX_EMAILS, so small and large n are
    both covered."""
    ratio = (MAX_EMAILS / MIN_EMAILS) ** (1 / (NUM_SIZES - 1))
    out = sorted({round(MIN_EMAILS * ratio ** i) for i in range(NUM_SIZES)})
    out[-1] = MAX_EMAILS
    return out

# ----- measuring -----

def run_program(program, input_path):
    """Run `echo input_path | program` and return (real, user, sys, max_rss_kb).

    The program reads the file *name* from stdin and prints emails as it runs,
    so its stdout and stderr are both thrown away. os.wait4 collects the same
    user/sys numbers `time` reports, plus the program's peak memory (max RSS).
    """
    start = time.perf_counter()
    proc = subprocess.Popen([program], stdin=subprocess.PIPE,
                             stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
    proc.stdin.write((input_path + "\n").encode())
    proc.stdin.close()
    _, status, usage = os.wait4(proc.pid, 0)
    real = time.perf_counter() - start
    proc.returncode = os.waitstatus_to_exitcode(status) # so Popen doesn't wait
    again
    if proc.returncode != 0:
        raise RuntimeError(f"{program} exited with code {proc.returncode} on
        {input_path}. "
                           f"Try it by hand: echo {input_path} | {program}")
    # ru_maxrss is kilobytes on Linux but bytes on macOS.
    max_rss_kb = usage.ru_maxrss / 1024 if sys.platform == "darwin" else
    usage.ru_maxrss

```

```

    return real, usage.ru_utime, usage.ru_stime, max_rss_kb

# ----- curve fitting (no numpy needed) -----

def fit_line(xs, ys):
    """Least squares for  $y = a + b \cdot x$ . Returns (a, b)."""
    mx, my = statistics.fmean(xs), statistics.fmean(ys)
    sxx = sum((x - mx) ** 2 for x in xs)
    if sxx == 0:
        return my, 0.0
    b = sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / sxx
    return my - b * mx, b

def sse(ys, preds):
    return sum((y - p) ** 2 for y, p in zip(ys, preds))

# Each model is  $y = a + b \cdot g(n)$ . The name is how it prints.
MODELS = {
    "log n": (lambda n: math.log(n), "math.log(n)"),
    "n": (lambda n: n, "n"),
    "n log n": (lambda n: n * math.log(n), "n * math.log(n)"),
    "n^2": (lambda n: n ** 2, "n ** 2"),
    "n^3": (lambda n: n ** 3, "n ** 3"),
}

def fit_models(ns, ys):
    """Fit every model and return a list of (aic, name, a, b, k, expr), best
    first.

    AIC penalizes extra parameters, so the free power law  $n^k$  only wins if it
    fits clearly better than the fixed shapes.
    """
    results = []
    m = len(ns)

    def aic(err, params):
        return m * math.log(max(err, 1e-300) / m) + 2 * params

    # Constant:  $y = a$ 
    a = statistics.fmean(ys)
    results.append((aic(sse(ys, [a] * m), 1), "1", a, 0.0, None, f"{a!r}"))

    for name, (g, expr) in MODELS.items():
        xs = [g(n) for n in ns]
        a, b = fit_line(xs, ys)

```

```

        if b < 0:
            continue # time and memory shouldn't shrink as input grows
        err = sse(ys, [a + b * x for x in xs])
        results.append((aic(err, 2), name, a, b, None, f"{a!r} + {b!r} *
{expr}"))

# Free power law:  $y = a + b * n^k$ , grid search over k
best = None
for i in range(10, 401):
    k = i / 100
    xs = [n ** k for n in ns]
    a, b = fit_line(xs, ys)
    if b < 0:
        continue
    err = sse(ys, [a + b * x for x in xs])
    if best is None or err < best[0]:
        best = (err, a, b, k)
if best:
    err, a, b, k = best
    results.append((aic(err, 3), f"n^{k:.2f}", a, b, k, f"{a!r} + {b!r} * n
** {k}"))

results.sort(key=lambda r: r[0])
return results

def r_squared(ns, ys, func):
    my = statistics.fmean(ys)
    ss_tot = sum((y - my) ** 2 for y in ys)
    ss_res = sse(ys, [func(n) for n in ns])
    return 1 - ss_res / ss_tot if ss_tot else 1.0

def report_fit(func_name, ns, ys):
    """Print the best-fitting model as a Python function and return its
    expression."""
    fits = fit_models(ns, ys)
    _, name, *_ , expr = fits[0]
    source = f"def {func_name}(n):\n    return {expr}\n"
    namespace = {"math": math}
    exec(source, namespace)
    r2 = r_squared(ns, ys, namespace[func_name])
    print(f"Best fit: 0({name})    R^2 = {r2:.4f}")
    print("Runner-ups: " + ", ".join(f"0({r[1]})" for r in fits[1:4]))
    print(source)
    return expr

# ----- main -----

```

```

def main():
    random.seed(210)
    for program in PROGRAMS:
        if not os.access(program, os.X_OK):
            raise SystemExit(f"Can't run {program}: make sure it exists here and
is executable.")

    rows = [] # (program, n, median real seconds, median max RSS in KB)
    with open(LOG_FILE, "w", newline="") as f:
        log = csv.writer(f)
        log.writerow(["program", "emails", "run", "real", "user", "sys",
"max_rss_kb"])
        for n in sizes():
            path = write_input(n)
            for program in PROGRAMS:
                reals, mems = [], []
                for run in range(1, REPEATS + 1):
                    real, user, sys_, mem = run_program(program, path)
                    log.writerow([program, n, run, f"{real:.6f}", f"{user:.6f}",
                                f"{sys_:.6f}", f"{mem:.0f}"])
                    f.flush()
                    reals.append(real)
                    mems.append(mem)
                real, mem = statistics.median(reals), statistics.median(mems)
                rows.append((program, n, real, mem))
                print(f"{program:12} n={n:>6} real={real:.3f}s mem={mem /
1024:.1f} MB")

    print(f"\nRaw measurements logged to {LOG_FILE}\n")

    fitted = {}
    for program in PROGRAMS:
        base = os.path.basename(program)
        ns = [n for p, n, _, _ in rows if p == program]
        times = [t for p, _, t, _ in rows if p == program]
        mems = [m for p, _, _, m in rows if p == program]
        print(f"=== {program}: time (seconds) ===")
        time_expr = report_fit(f"{base}_time", ns, times)
        print(f"=== {program}: peak memory (KB) ===")
        mem_expr = report_fit(f"{base}_memory_kb", ns, mems)
        fitted[program] = (ns, times, mems, time_expr, mem_expr)

    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("(Install matplotlib to also get a plot: pip install matplotlib)")
        return
    fig, (ax_time, ax_mem) = plt.subplots(1, 2, figsize=(12, 5))

```

```

    smooth = list(range(MIN_EMAILS, MAX_EMAILS + 1, 50))
    for program, (ns, times, mems, time_expr, mem_expr) in fitted.items():
        for ax, ys, expr in ((ax_time, times, time_expr), (ax_mem, mems,
mem_expr)):
            func = eval(f"lambda n: {expr}", {"math": math})
            line, = ax.plot(ns, ys, "o", label=f"{program} measured")
            ax.plot(smooth, [func(n) for n in smooth], "-",
color=line.get_color(),
                    label=f"{program} fit")
            ax_time.set(xlabel="emails", ylabel="real time (s)", title="Time")
            ax_mem.set(xlabel="emails", ylabel="peak memory (KB)", title="Space")
            ax_time.legend()
            ax_mem.legend()
            fig.tight_layout()
            fig.savefig("timings.png", dpi=150)
            print("Plot saved to timings.png")

if __name__ == "__main__":
    main()

```

This is the modified version of the original program, which analyzes the space as opposed to time complexity.

The output for Sonnet was:

```

=== ./anthropic: peak memory (KB) ===
Best fit: 0(1)    R^2 = 1.0000
Runner-ups: 0(log n), 0(n), 0(n log n)
def anthropic_memory_kb(n):
    return 13312.0

```

while the Kimi model churned out the same

```

=== ./moonshot: peak memory (KB) ===
Best fit: 0(1)    R^2 = 1.0000
Runner-ups: 0(log n), 0(n), 0(n log n)
def moonshot_memory_kb(n):
    return 13312.0

```

It's the same as Kimi's linked list implementation, but even more dramatic. For all the theoretical, there really is not too much difference to do a heap allocation array or a linked array in terms of speed, as passing around references requires the operating system to allocate memory, which it still does in blocks itself. It makes no significant difference.

Indeed, operating systems handle memory allocation very liberally. Instead of handing memory on a need-to-have basis, operating systems often hand programs more memory than they ask for, so future OS calls for more memory are handled by just giving it more of that memory. This is better in the long run because programs usually have many heap calls, and this lets the OS

handle memory in less stress. Plus, a couple of kilobytes being passed around has little cost for performance and active memory.

Therefore, when Kimi allocates a Node, the OS itself may actually reserve the space for more nodes. When Kimi, inevitable, tries to allocate another node, memory already marked off for the process that sits unused is handed to Kimi. In the end, this means that there is no significant difference between reallocating the heap pointer to have twice the size or just doing it element by element, because the actual space the operating system will use ends up being identical between both processes. So both of their space complexities end up being the same, but Kimi still suffers from the horrid time complexity.

Maintainability

Maintainability in terms of a program like this is interesting. I will try my best to understand C++ standards and conventions, but note that I am learning this language and writing this assignment concurrently.

In terms of raw abstract implementation, Sonnet easily gets the win on this. In any basic DSA class (or Programming II in KU) using list/array implementations for maxheaps is just the unbeatable standard. It implements the same functionality as using nodes, but tacitly and more efficiently. I myself, trying to work ahead on the labs, quickly learned that the MaxHeap implementation of the labs was severely lacking and difficult to abstractly work with. I cannot give Kimi anything but a failure in this abstract regard.

There are subtle things I would use in C that probably translate to C++. An example is Sonnet's insistence on using return statements:

```
// higher rank = read sooner
static int RankOf(const std::string& category) {
    if (category == "Boss")           return 5;
    if (category == "Subordinate")    return 4;
    if (category == "Peer")           return 3;
    if (category == "ImportantPerson") return 2;
    if (category == "OtherPerson")    return 1;
    return 0; // unknown
}
```

instead of using an enum that matches towards the strings. However, these are small nitpicks. C++'s string library is also a lot more robust than C (which is apples to oranges, but still of note). For example, the way it is able to store the date as a string and quickly and efficiently use `static_cast` to return the long version of the data is elegant.

There is a little premature optimization, for example with long long sequence. This is used in Sonnets to break ties, but likely doesn't need to be long long unless we handle billions of emails. Still, it is a small cost.

std::Vector and C versus C++

This wouldn't be a C++ analysis to say that it would probably be a good idea to use a `std::vector`, because of how it facilitates memory management. However, the explicit directions given by our prompt disallow any pre-existing heap modules. This doesn't mean no vectors, though. I theorize that the reason that Sonnet, at least, didn't attempt to include Vectors is because it was worried about that being considered a "heap module", in some sense. But it's just a basic part of the standard library to facilitate heap-allocated arrays and make them simple. On the contrary, I think the choice to forcibly use arrays in a heap allocated sense defers to C-like syntax and in fact makes the program worse C++ and less maintainable. The implementation, besides some idiomatic conventions and so, actually smell more of C, similar to its initial implementation in Assignment 2. Kimi circumvents that by implementing a (worse) node-base maxheap program.

Because of our vector abstinence, we lose all sorts of useful properties, such as a built in destructor. We also get contiguous heap allocated members with self contained destructors. dirkgently puts it well for us:

<https://stackoverflow.com/questions/849168/are-stdvector-elements-guaranteed-to-be-contiguous>

The elements of a vector are stored contiguously, meaning that if `v` is a vector where `T` is some type other than `bool`, then it obeys the identity `&v[n] == &v[0] + n` for all `0 <= n < v.size()`.

Vectors are a perfect and adequate way to store our elements. However, I think that both LLMs tried to abstract based on older, C style examples, and both produced vastly inferior results.

Noteworthy C++isms

This is in maintainability because it confused me as a C developer never having used C++.

```
//moonshot.cpp, line 136
--count; //why not count--;?
```

The reason for not doing `count--`; is because of how both C and C++ use ordering. As you likely know:

```
int a, b;
a=5=b; // a= 5 returns 5 by value, which sets b to be 5
```

So we can idiomatically assume we don't deincrement a value and use it by calling the prefix order instead of the suffix (`-a` instead of `a-`)

This seems like such a random thing to enforce, but I've seen both KIMI and Sonnet do it every single time they deincrement a variable. Perhaps better to be safe than sorry.

C++'s string conversion library is scary.

```
Date(const string& s) {
    month = stoi(s.substr(0, 2));
```

```

    day   = stoi(s.substr(3, 2));
    year  = stoi(s.substr(6, 4));
}

Email(const string& payload) {
    size_t c1 = payload.find(',');
    size_t c2 = payload.rfind(',');
    sender   = payload.substr(0, c1);
    subject  = payload.substr(c1 + 1, c2 - c1 - 1);
    date     = Date(payload.substr(c2 + 1));
}

```

The conversion is clunky, but it works. From a superset of C, I wasn't expecting better, but the actual solution is much simpler than I thought. The only problem is error handling, which is kind of not really processed on payload. There can be malformed data, and the conversion is not the best because of that.

It's strange because LLMs promote really safe C++ until it conflicts with the "blueprint" (i.e. the implementation of the code it is recycling from some other code it has stored), and then does strange things. There's no way to determine an invalid email in its actual creation, and the error catches implicitly in other parts of the code. I find this bizarre, but understandable.

Observations on Maintainability and The Diegetic Prompt

The core issue coming from the maintainability is that the implementation is so based on the wording given. The LLM is great at making the code in a great volume, but is finicky with all sorts of parts. Ultimately, we see this lead to an ultra-literal translation of the user's prompt, resulting in a flawed implementation. We can be as idiomatically descriptive to each part of the program (ex. "use a Vector so you don't have to deallocate your own array") but there lies the problem: in order to achieve the intended features for our project, implemented in a reasonable way, we need to specifically identify what we want for every step, and to understand that, we need to understand the terms that the computer understands to optimize it (such as "Vector based list" instead of "Max Heap", as both LLMs use varying unoptimal parts). This, combined with the scaling of a large project, leads us to a strange paradox: LLM are great at optimizing writing large amounts of code, but they don't write perfect code, and need to be specifically prompted to. That prompting can't be done with the LLM as the LLM makes the same mistakes when writing the code as thinking through the code (indeed, Sonnet and Kimi turn the prompt into instructions and recursively achieve each "goal" or task by using tokens to process it, but this process is flawed in having a perfect implementation). So if we arrive at the natural conclusion that "a perfect language prompt specifies every part of the program" we have reinvented programming, in a non-deterministic language implementation.

Of course, this does not mean that LLMs themselves are bad at writing all code. But for an LLM to write "perfect" code (code that optimally fits the specification) it needs to be prompted specifically by a human or have its output scrutinized by a human, both requiring human input. Maybe transformers will become better at generating this code, as we have seen before. But for the time

right now, I do not think this output from one prompt is adequate enough of a solution to be implemented “well” or maintainably, by both models, even if the response was quick.

Human Revision

I decided to prefer Sonnet’s implementation. Sonnet does a better job at actually implementing a normal maxheap, even if it defaults to C like structures. Kimi’s implementation is bizarre and much slower. The main optimization i’ll be doing is implementing vector based arrays to make everything generally faster and more C++ like.

Since I know exactly what the issue is, and it’s a small change, I used Claude’s Opus 5.5 to pair program this feature. I mostly did this to avoid having to write C++, but it also saved time because I knew what to look for. I limited Opus 5.5’s scope to just improving the Vector system, and carefully annotated its code with my own comments and revision. Below is that result:

```
// ceo_inbox.cpp
// EDITED BY LUCAS FRIAS
// REVISIION GENERATED ** BY CLAUDE OPUS 5.5 **
// wanna be as explicit as i can. i did not hand program this
// revision comments are below:
// - added comments and fixed code to use vectors.
// - sender category is now a uint8_t enum instead of a string.
// prioritizes emails for a ceo using a hand-built max heap.
// build: g++ -std=c++11 -Wall -Wextra -o ceo_inbox ceo_inbox.cpp
// run:    ./ceo_inbox inbox.txt

#include <stdint>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// -----
// category: who sent the email. the value is the rank.
// higher rank = read sooner.
// -----

//uses an enum to tstore the category, explicitly sets each value
enum class Category : std::uint8_t {
    Unknown          = 0,
    OtherPerson       = 1,
    ImportantPerson   = 2,
    Peer              = 3,
    Subordinate       = 4,
    Boss              = 5
};

// -----
```

```

// email: one message. knows how to rank itself.
// -----
class Email {
private:
    Category    sender; //uses our enum
    std::string subject; //subject still has to be a string
    std::string date;    // MM-DD-YYYY
    long long    sequence; // arrival order, breaks exact ties (earlier wins)

    // MM-DD-YYYY -> YYYYMMDD so a bigger number means a newer date
    static long DateKeyOf(const std::string& d) {
        //given a constant string slice reference converts the date into
        //a long, is a static function
        if (d.size() != 10 || d[2] != '-' || d[5] != '-') return 0;
        //error handling for incorrect date, return a simple zero
        int month = std::stoi(d.substr(0, 2)); //convert the month by the first
0-2 chars
        int day   = std::stoi(d.substr(3, 2)); //next is the day, we american here
        int year  = std::stoi(d.substr(6, 4)); //lastly the year
        return static_cast<long>(year) * 10000L + month * 100L + day; //returns
the formatting
    }

public:
    Email() : sender(Category::Unknown), subject(""), date(""), sequence(0) {} //
inits a default email type for its constructor
    //we don't need a destructor because this is just a normal object interface.
this goes out of scope with
    //our vector now
    Email(Category s, const std::string& subj, //defines our init method and
parameters
        const std::string& d, long long seq)
        : sender(s), subject(subj), date(d), sequence(seq) {}

    //these functions will return the sender, subject, date, etc for every
    //category and value that we receive here
    Category    GetSender() const { return sender; }
    const std::string& GetSubject() const { return subject; }
    const std::string& GetDate()    const { return date; }

    //this will return the category, these are the only defined category and
    //will do string matching with our enum
    static Category ParseCategory(const std::string& text) {
        if (text == "Boss")          return Category::Boss;
        if (text == "Subordinate")    return Category::Subordinate;
        if (text == "Peer")           return Category::Peer;
        if (text == "ImportantPerson") return Category::ImportantPerson;
        if (text == "OtherPerson")    return Category::OtherPerson;
        return Category::Unknown; //default fall through case
    }

```

```

}
//returns the category to a string so we can display it, but we use cstrings
//because it will be stored in BSS as a string literal (that's why
//we're not using C++ string types)
static const char* CategoryName(Category category) {
    switch (category) {
        case Category::Boss:           return "Boss";
        case Category::Subordinate:     return "Subordinate";
        case Category::Peer:           return "Peer";
        case Category::ImportantPerson: return "ImportantPerson";
        case Category::OtherPerson:     return "OtherPerson";
        case Category::Unknown:         break;
    }
    return "Unknown";
}

//determines whether a given date is a valid date
//given its string slice, returns a boolean if true or false
//doesn't actually determine whether or not the date is a real
//date like could exist on the calendar, more like if it's a real
//date like it could be formatted in the way a real date
static bool IsValidDate(const std::string& d) {
    //check the size of the string, and that it's all dashed like XX-XX-XXXX
    if (d.size() != 10 || d[2] != '-' || d[5] != '-') return false;
    //now we iterate
    for (std::size_t i = 0; i < d.size(); ++i) {
        //skip the - for the second and fifth spot in the i
        if (i == 2 || i == 5) continue;
        //if it's not in the range of ascii chars to be considered valid
return false
        if (d[i] < '0' || d[i] > '9') return false;
    }
    //return true
    return true;
}

// true when this email must be read before the other one
bool HasHigherPriorityThan(const Email& other) const {
    //determine whether it has a priority
    //if the senders aren't equal default to left, versus right
    //false being left, right being right
    if (sender != other.sender) return sender > other.sender;

    long myDate    = DateKeyOf(date); //gets the date, through the DateKeyOf
    long theirDate = DateKeyOf(other.date); //gets theirDate
    if (myDate != theirDate) return myDate > theirDate; //checks which is
closer or not, since these are the
    //same category of senders

```

```

        return sequence < other.sequence; //earliest arrival
physically based on Sonnet's tiebreaking system
    }
};

// -----
// maxheap: vector-based binary heap. built from scratch.
// parent of i is (i-1)/2. children of i are 2i+1 and 2i+2.
// -----
class MaxHeap {
private:
    std::vector<Email> items; //makes a vector based email
    //vector implements construction, destruction, etc. this is fully implemented
already
    //by the library which makes it easier

    void Swap(std::size_t a, std::size_t b) {
        //swap the left and right elements as so
        Email temp = items[a];
        items[a] = items[b];
        items[b] = temp;
    }

    void SiftUp(std::size_t index) {
        //upheap, basically the same as we'd do it for an array, we just happen
to
        //have a dynamic array which feels pretty nice

        while (index > 0) {
            //while we're not at the top
            std::size_t parent = (index - 1) / 2; //get what our parent is
            if (items[index].HasHigherPriorityThan(items[parent])) {
                //if we're a bigger priority than the other email (which depends
on sender, etc)
                //also we access the items index part
                Swap(index, parent); //swap the two
                index = parent; //set it
            } else {
                break; //the element is where it should be, we can break
            }
        }
    }

    void SiftDown(std::size_t index) {
        //we sift down and set an element to a lower priority
        while (true) {
            //lets calculate the left, right, and assume that the "largest"
            //is the index, where we currently are iterating
            std::size_t left = 2 * index + 1;

```

```

        std::size_t right = 2 * index + 2;
        std::size_t largest = index;
        //is the left the bigger one? then set it as the largest
        if (left < items.size() &&
items[left].HasHigherPriorityThan(items[largest]))
            largest = left; //set it here
        if (right < items.size() &&
items[right].HasHigherPriorityThan(items[largest]))
            largest = right; //do the same with the right, set the values here

        if (largest == index) break; //the largest is the index we're at
right now, we don't need to sift up
        Swap(index, largest); //swap the current index with the largest, and
set the index to the largest
        index = largest; //index is now the largest
    }
}

public:
    //public functions
    //returns whether or not we are empty and gets our size through accessing
items
    bool IsEmpty() const { return items.empty(); }
    std::size_t Size() const { return items.size(); }

    //we just add the element and sift up given its index.
    //since we sift up at its index we are able to actually
    //to promote it to the highest value it needs to be
    void Insert(const Email& email) {
        items.push_back(email); //push the email back to the end
        SiftUp(items.size() - 1); //sift the elements up
    }

    // caller must check IsEmpty() first
    const Email& Peek() const { return items[0]; }

    // caller must check IsEmpty() first
    Email ExtractMax() {
        //let's first get the top
        Email top = items[0];
        //now we set the top element to the lowest element we have
        //the reason for this is we need to preserve the exact order
        //and this is the best implementation to ensure that all values get
sorted from the top
        //down to their exact order by kinda iterating through the index
        items[0] = items.back();
        items.pop_back(); //pop_back, remove the items
        if (!items.empty()) SiftDown(0); //if it's not empty we sift down from the
top which now has the lowest value
    }

```

```

        return top;//return the top when we're done
    }
};

// -----
// inboxprocessor: owns the heap, reads the command file, prints results.
// -----
class InboxProcessor {
private:
    //
    MaxHeap    heap;
    long long  nextSequence;

    //returns the string trimmed, just removes the whitespace
    static std::string Trim(const std::string& text) {
        //this string contains all the whitespace,
        const std::string whitespace = " \\t\\r\\n";
        //gets the first of all whitespace
        std::size_t first = text.find_first_not_of(whitespace);
        //if it's an empty string stop here
        if (first == std::string::npos) return "";
        //gets the last of all whitespace
        std::size_t last = text.find_last_not_of(whitespace);
        //return the replacement of all whitespace before and after
        return text.substr(first, last - first + 1);
    }

    void HandleEmail(const std::string& fields) {
        //firstComma, find the ","
        std::size_t firstComma = fields.find(',');
        std::size_t secondComma = (firstComma == std::string::npos
                                   ? std::string::npos
                                   : fields.find(',', firstComma + 1));
        //second comma, same thing, we're just trying to find the next occurrence
        if (secondComma == std::string::npos) {
            //output an error if we get the null terminator char
            std::cerr << "skipped bad EMAIL line (need 3 fields): " << fields <<
            "\n";
            return;
        }
        //okay now we have to parse all of the strings at their position
        //to get them before we translate it, this is not the most exciting
        //work we just process after the indices
        std::string senderText = Trim(fields.substr(0, firstComma));
        std::string subject    = Trim(fields.substr(firstComma + 1,
                                                    secondComma - firstComma -
1));
        std::string date      = Trim(fields.substr(secondComma + 1));
    }
};

```

```

        //now we get the sender by parsing the category senderText
        Category sender = Email::ParseCategory(senderText);
        if (sender == Category::Unknown) {
            //remvoe the default "unknown" sender when the enum string eval falls
            through
            //kinda makes some of the handling dead code but this could be
            triggered
            std::cerr << "skipped EMAIL with unknown sender: " << senderText <<
            "\n";
            return;
        }
        if (!Email::IsValidDate(date)) {
            //if the email has an invalid date, skip it
            std::cerr << "skipped EMAIL with bad date: " << date << "\n";
            return;
        }
        //otherwise throw it in the heap, using the suffix so we can get
        nextSequeucenand tierate
        //it as well
        heap.Insert(Email(sender, subject, date, nextSequence++));
    }

    void HandleCount() const {
        //prints the string of handlecount, and displays the emails left to read
        std::cout << "There are " << heap.Size() << " emails to read.\n\n";
    }

    void HandleNext() const {
        //if the heap is empty, then there are no emails left to read
        if (heap.IsEmpty()) {
            std::cout << "No emails to read.\n\n";
            //returns the emails left to read
            return;
        }
        const Email& top = heap.Peek();
        //get what the top is
        //display the output
        std::cout << "Next email:\n"
            << "Sender: " << Email::CategoryName(top.GetSender()) << "\n"
            << "Subject: " << top.GetSubject() << "\n"
            << "Date: " << top.GetDate() << "\n\n";
    }

    void HandleRead() {
        //handle the email being read, and if its empty, display nothing
        if (heap.IsEmpty()) {
            std::cout << "No emails to read.\n\n";
            return;
        }
    }

```

```

        heap.ExtractMax();//otherwise extract the top value
    }

public:
    //make an inbox processor that contains the heap
    InboxProcessor() : heap(), nextSequence(0) {}

    bool ProcessFile(const std::string& path) {
        //tries to process our file, returns true
        //or false depending on its value

        //the ifstream input
        std::ifstream input(path.c_str());
        //if its not open err out and return false
        if (!input.is_open()) {
            std::cerr << "cannot open file: " << path << "\n";
            return false;
        }

        //get the sttring line
        std::string line;

        //while we get each input, line
        while (std::getline(input, line)) {
            line = Trim(line);//trim whitespace on the line
            if (line.empty()) continue;//ignore empty lines

            std::size_t space = line.find(' '); //find the spce
            std::string command = (space == std::string::npos) ? line
                                                                    : line.substr(0,
space); //get the command,
            std::string rest = (space == std::string::npos) ? ""
                                                            :
line.substr(space + 1); //then the rest follow after the first space as per the
format

            //this matches the string with the current function for every value,
            //corresponds with string literals. sends an error message if they
don't know
            if (command == "EMAIL")    HandleEmail(rest);
            else if (command == "COUNT") HandleCount();
            else if (command == "NEXT") HandleNext();
            else if (command == "READ") HandleRead();
            else std::cerr << "skipped unknown command: " << line << "\n";
        }
        return true;
    }
};

```

```

// -----
// application: entry point wrapper so main holds no logic.
// -----
class Application {
public:
    int Run(int argc, char* argv[]) {
        //run the main application, very object orientated c++
        std::string path;
        //get the string path

        //check if we passed any string, if so that's our path
        if (argc > 1) {
            path = argv[1];
        } else {
            //otherwise prompt the user for the recommended input file
            std::cout << "input file: ";
            std::getline(std::cin, path);
        }

        InboxProcessor processor; //make an inboxprocessor
        //run the main process file, returna boolean depending on the actual
        //succes state
        return processor.ProcessFile(path) ? 0 : 1;
    }
};

int main(int argc, char* argv[]) {
    //last but not least, our main function.
    //we probably don't need the App wrapper but it's
    //very principled so i'll leave it
    Application app; //make an app
    return app.Run(argc, argv); //return the returned return statement from our
    email processor -> application -> main function
}

```

The main changes are the implementation of having vectors and enums in this thing.

Vectors are fairly simple, it just reduces code overhead for creating and allocating every method. We can just push a new value to the vector, which automatically adds it to the end of the vector/array automatically. We don't have to worry about destructing elements because we just peek from the top, which pops off and allows us to inheap.

The enum is an optimization that I implemented before, it's just a generic part of the implementation as both LLMs expect it to be a "sender" (someone with a specific email address) instead of a category of sender. A uint8_t is much smaller and easier to deal with than an extra heap allocated string, and is more optimal for this solution.

Using the time and space complexity analysis Python script from before, we can get this output:

```

=== ./anthropic: time (seconds) ===
Best fit:  $O(n)$   $R^2 = 0.9939$ 
Runner-ups:  $O(n^{0.96})$ ,  $O(n \log n)$ ,  $O(n^2)$ 
def anthropic_time(n):
    return 0.0019066916748283914 + 2.092385776885594e-06 * n

=== ./anthropic: peak memory (KB) ===
Best fit:  $O(1)$   $R^2 = 1.0000$ 
Runner-ups:  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$ 
def anthropic_memory_kb(n):
    return 13312.0

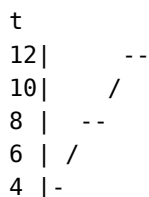
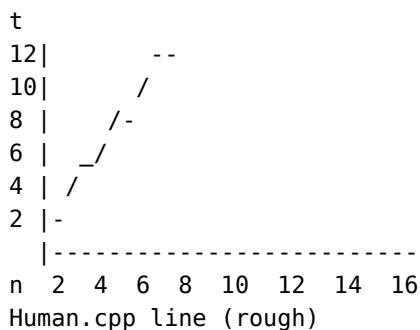
=== ./human: time (seconds) ===
Best fit:  $O(n)$   $R^2 = 0.9980$ 
Runner-ups:  $O(n^{0.98})$ ,  $O(n \log n)$ ,  $O(n^2)$ 
def human_time(n):
    return 0.0015497823530943914 + 1.4918856773748935e-06 * n

=== ./human: peak memory (KB) ===
Best fit:  $O(1)$   $R^2 = 1.0000$ 
Runner-ups:  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$ 
def human_memory_kb(n):
    return 13312.0

```

We can see slightly overall improvements in the time complexity for each program in terms of small n values. By just using heap vectors and enums, our slope scales slightly lower, from 2.09 > 1.49. We have a decent optimization from just understanding the basic C++ attributes and methods of running these programs. This input is generally optimized in a linear sense by a huge time factor.

In fact, by graphing the slopes directly (which represents the time in less than nanoseconds) we can see there is a general linear trend that grows for massive chunks of data



```

2 |
  |-----
n  2  4  6  8 10 12 14 16
Human.cpp line (rough)

```

The slope is much more gradual and has a linear scaling difference for large input sizes. By implementing vectors instead of doing it ourself through the abtsraction of the class, we save time, make our program safer, and make it more maintainable and simple

Conclusion

I appreciate your review of this assignment, as always. I hope this was an interesting comparison between the Chinese and American models and some interesting implementations. LLMs always have varying results, and I wonder how biased these results are, but Sonnet seems much better at “oneshot” prompts that this assignment requires. Even still, there are sma;; things that make this code interesting bt at any sort of large scale, unmaintainable.

I hope you enjoyed reading through this analysis. Again, C++ is not my favorite language and I am not the best at it, but I hope that my analysis was thorough. Have a great rest of your day.

Lucas Frias